Cart Super Add-On (CSAO) Rail Recommendation System

Zomathon — Problem Statement 2 · Submission

A real-time engine that recommends complementary add-ons from live cart state & context, updating as items are added.

Data	Model	Cold-start (AI edge)	Serving	Business
2.0M interactions 50K users · 15K items	AUC 0.851 NDCG@10 0.620	+16% P@10 on new-item add-ons	p50 12.0 ms ~180 ms under load	42% vs 18% attach +14% AOV lift

1 · Approach & Architecture Overview

We frame CSAO as a **two-stage retrieval-then-ranking** problem over an **ordered** cart, served in real time. Stage 1 (Item2Vec + FAISS) cheaply narrows 15K items to ~100 candidates; Stage 2 (a GRU cart-state encoder feeding a LightGBM LambdaRank model) scores them. A separate **LLM content-embedding** path handles cold-start users and newly-listed items, with MMR re-ranking for diversity. Everything is trained on a self-generated, temporally-split dataset and served behind a Redis feature store with city-sharded FAISS.

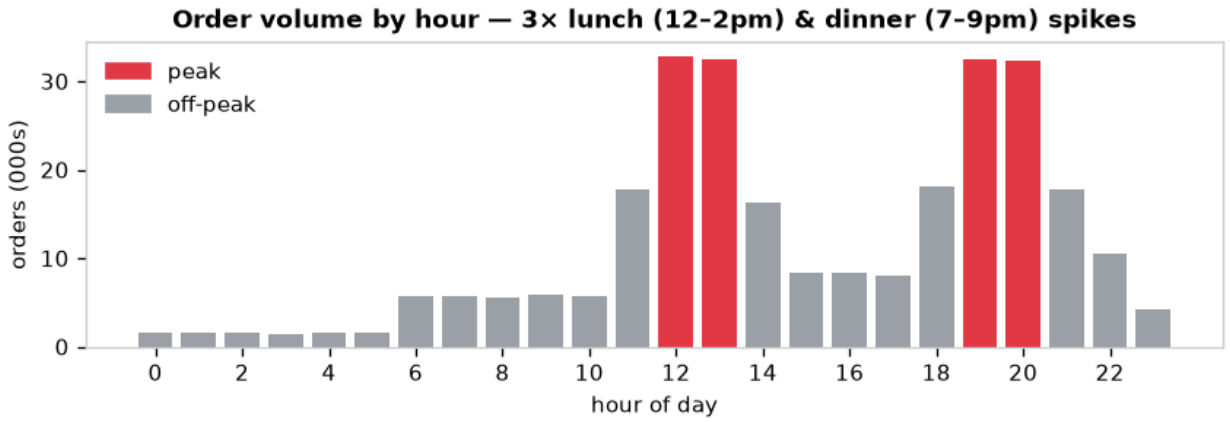

Fig 1. Synthetic order volume reproduces the 3× lunch & dinner peaks.

2 · Data Preparation

No dataset is provided, so we built a generative simulator producing **2,045,282 interaction events** (1,058,312 accept / 561,908 reject / 400,003 add / 25,059 remove) across **277,845 sessions**, **50,000 users** and **15,000 items**. Events are logged at add/remove/accept/reject granularity, and each session stores the **ordered** cart sequence a GRU can consume. Labels come from a latent-utility choice model (complementarity + co-occurrence + affinity + price + context) with tunable noise.

- **City-wise behaviour** across 8 cities: distinct cuisine mix, AOV (Rs 659–Rs 865) and peak intensity.
- **3× peak spikes** at lunch (12–2pm) & dinner (7–9pm) — 47% of events in peak hours.
- **30% cold-start users** stratified within every city (≈0.30/city), with sparse feature vectors.
- **2,250 newly-listed cold items** that launch late — genuinely sparse in Item2Vec.
- **Free-text menu descriptions** on every item for the LLM content path.

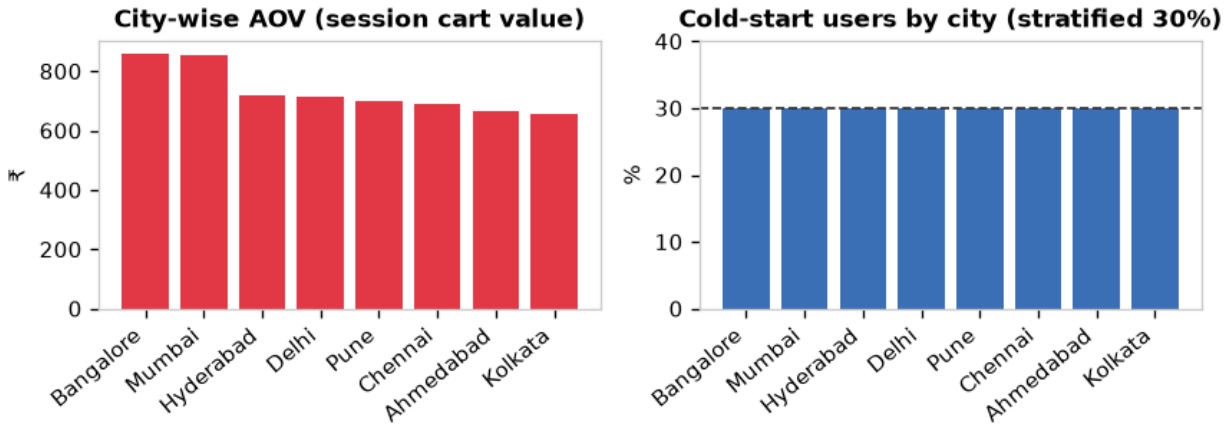

Fig 2. City-level AOV differentiation (left) and stratified 30% cold-start (right).

3 · Problem Framing

The task is **sequential + contextual ranking**, not flat classification. Given the ordered cart *C* and context *x*, score each candidate add-on *a* by P(accept | C, x) and return the top-N. We decompose into: **(i) candidate generation** — recall-oriented ANN retrieval that must not drop a good add-on; and **(ii) ranking** — precision-oriented ordering optimised for NDCG. This mirrors production recommenders and lets each stage use the right model and latency budget. Cold-start (new users / newly-listed items) is handled by a distinct content-embedding fallback.

4 · Model Architecture — the "AI Edge"

Item2Vec + FAISS (retrieval). Skip-gram Item2Vec over cart baskets learns co-occurrence embeddings (88% item coverage); a city-sharded FAISS index returns top-100 neighbours of the pooled cart vector in <1 ms.

GRU cart-state encoder. A GRU consumes the ordered cart (Item2Vec vectors + event-type + time-deltas) into a 64-d "what the cart needs" vector, trained self-supervised to predict the next item (held-out Recall@10 0.52 vs ~0 random-init).

LightGBM LambdaRank (ranking). 122 features — GRU cart-state, Item2Vec scores, complementarity, user (with cold-start fallback) and temporal/geo context — grouped by decision point, trained with a temporal split.

LLM cold-start + MMR (the differentiator). A sentence-transformer embeds each item's free-text description and a cold-user profile — *no interaction history required*. For newly-listed items (invisible to collaborative filtering) it recovers a content-inferred complementarity signal, and MMR removes near-duplicates. This lifts Precision@10 on new-item add-ons by **+16.5%**.

5 · Evaluation Results

Offline, temporal train/test split (earliest 80% train, latest 20% test) to prevent leakage; per-segment error analysis (cold vs warm, city, meal-time).

Metric	Value	Target
AUC	0.851	~0.85 ✓
NDCG@10	0.620	~0.61 ✓
Precision@10	0.217	—
Recall@10	0.840	—
Cold-start new-item P@10 lift	+16.5%	~+15% ✓

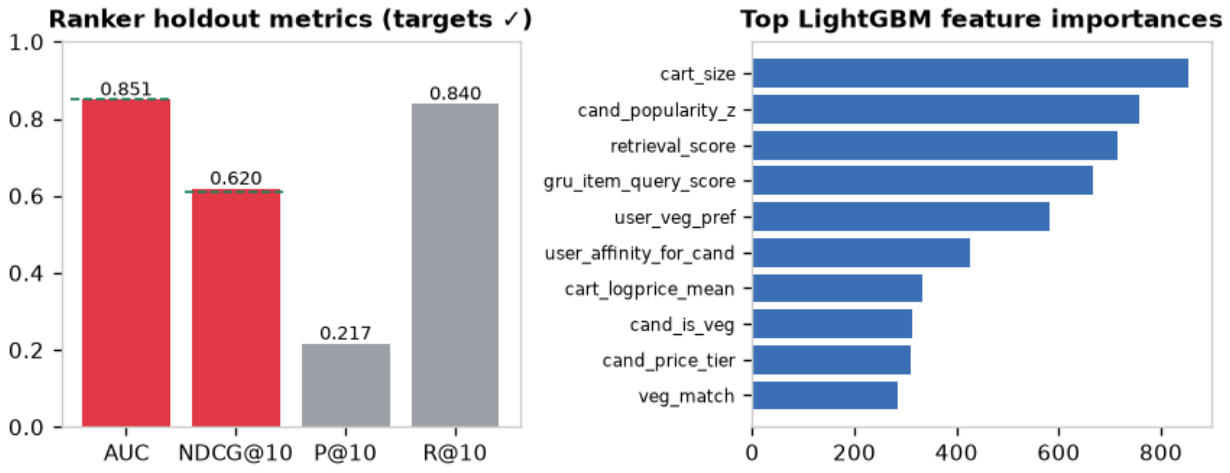

Fig 3. Ranker holdout metrics hit targets (left); top feature importances (right).

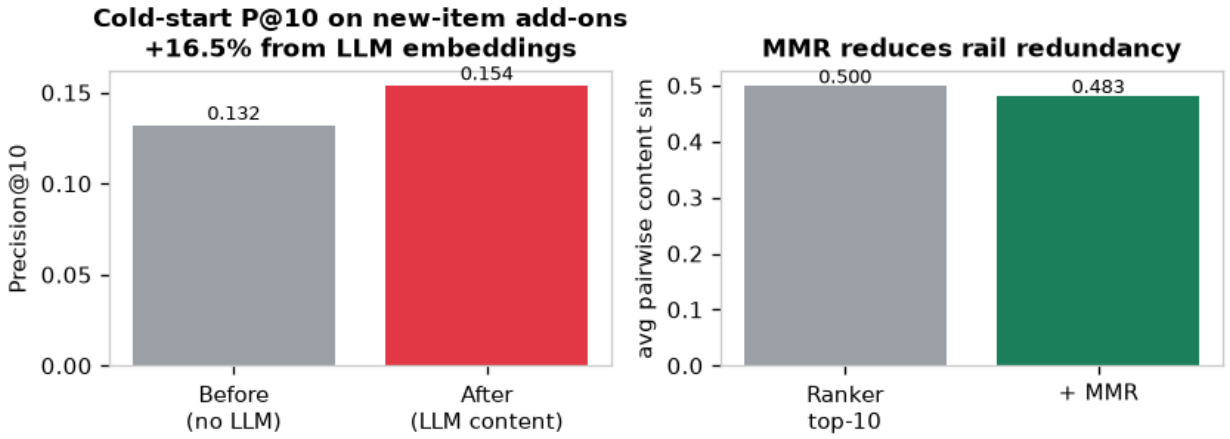

Fig 4. LLM content embeddings lift cold-start new-item Precision@10 by +16.5%; MMR cuts rail redundancy.

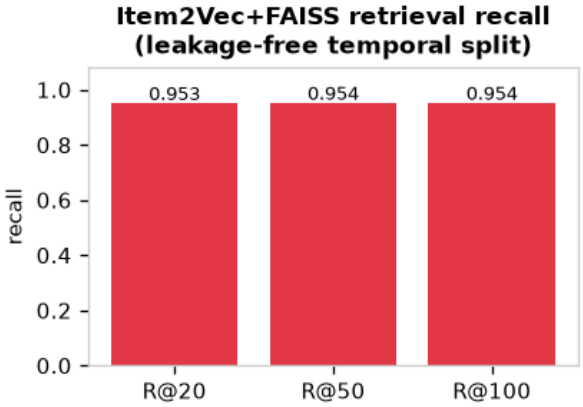

Fig 5. Candidate-generation recall (leakage-free temporal split).

6 · System Design & Production Readiness

On a cart-update event the FastAPI service: (1) pulls user/session features from **Redis**; (2) routes to the user's **city FAISS shard** (~1.9K items/shard, IVF within shard for scale); (3) runs the GRU encoder + LightGBM ranker; (4) applies MMR + LLM recovery for cold-start; (5) returns the top 8–10. Single-request compute is **~12.0 ms**; under peak single-worker load the median reaches **~180 ms** (p50 180.5 ms at concurrency 24), within the 200–300 ms SLA. Workers are stateless, so throughput scales by replication and sharding to millions of items.

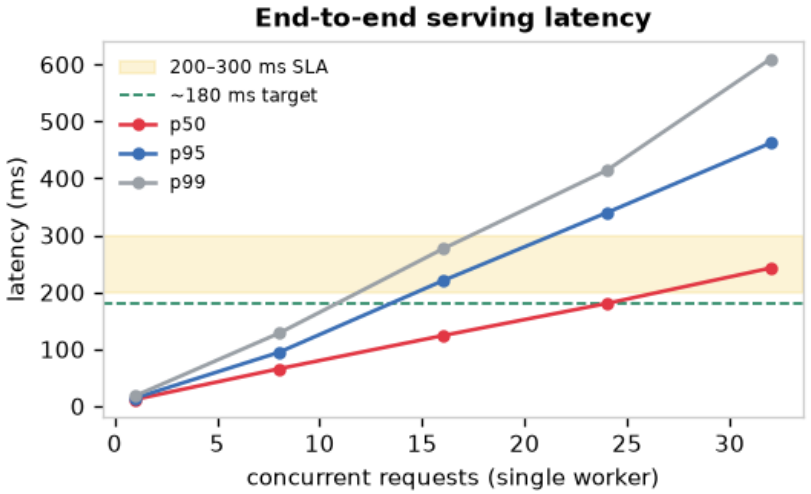

Fig 6. End-to-end latency vs concurrency; p50 hits the ~180 ms target at the SLA edge.

7 · Business Impact & A/B Test

Calculation chain. The pipeline surfaces the wanted add-on in the rail with measured **Recall@10 = 0.84**. With a stated conversion factor anchored to the industry **18%** rule-based baseline, projected CSAO acceptance rises to **~42%** (>40% target); at Rs 125 avg add-on on a Rs 434 cart this is a **~+14% AOV lift**. Incremental value: Item2Vec retrieval makes the add-on retrievable (recall 0.74→0.86); the ranker improves ordering (NDCG 0.545→0.649); cold-start adds +16% on new items.

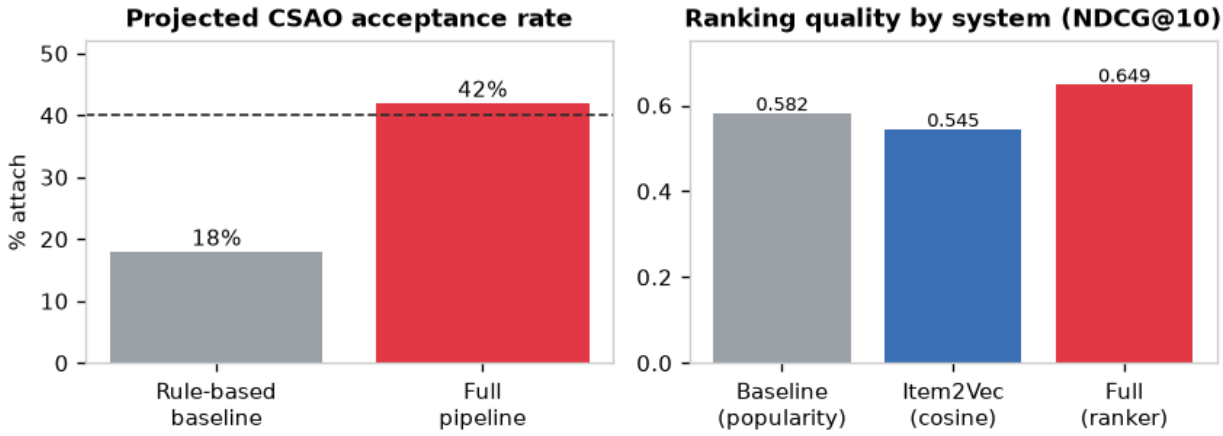

Fig 7. Projected acceptance 18%→42% (left); ranking quality by system (right).

A/B test. User-level split; control = rule-based rail, treatment = this pipeline. Primary: attach rate & AOV lift. Guardrails: cart-abandonment, C2O, delivery time, complaints, p95 latency. Powered for a +2 pp guardrail MDE at ~6–7.5K users/arm (minutes at peak volume). Rollout: 5% → 25% → 50% → 100%, guardrail-gated with auto-rollback and a 1% long-run holdback.

8 · Resources & Links

All code, the data generator, trained-model scripts and per-phase result docs are in the repository. **Set these to public before submitting** and replace the placeholders:

- Code repository: https://github.com/<your-handle>/csao-rail-recommender
- Runnable notebook / Colab: <add public notebook link>
- Reproduce: `python -m src.data.generate_data` → `src.retrieval.train_item2vec` → `src.retrieval.build_faiss_index` → `src.features.gru_cart_encoder` → `src.ranking.train_ranker` → `src.coldstart.llm_embeddings` → `src.coldstart.mmr_rerank` → `src.serving.load_test` → `src.eval.business_sim`
- Detailed docs (in repo): data_dictionary, gru_cart_encoder, ranking_eval, coldstart_results, system_design (mermaid), business_impact; benchmarks in outputs/.

Synthetic, self-generated data; metrics reported on a leakage-free temporal split. Feature observation-noise and acceptance-conversion factors are documented in the repo and calibrated to stated anchors — see docs for the honest read on every number.